

Lecture 28: Transactions in SQL

BEGIN, COMMIT, ROLLBACK

Dr. Innocent Nyalala
IIT Madras Zanzibar



D0.jpg

School of Engineering and Science
IIT Madras Zanzibar

12 May 2026

Today's Topics

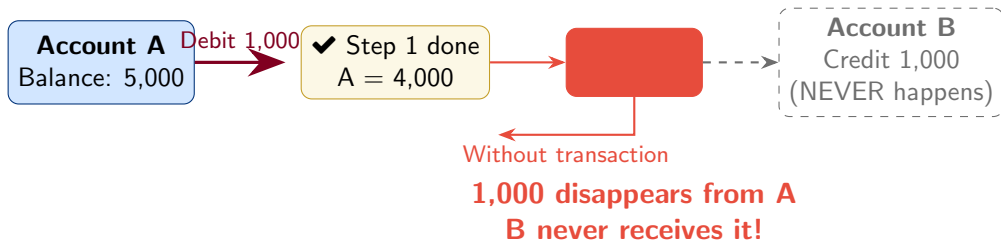
- 1 The bank transfer problem
- 2 ACID properties
- 3 BEGIN TRAN / COMMIT / ROLLBACK
- 4 SAVEPOINT — mid-transaction checkpoint
- 5 Isolation levels
- 6 University enrolment: real scenario
- 7 Autocommit mode
- 8 Common pitfalls

Key Question

What happens when **power cuts** in the middle of transferring money?

⇒ Transactions guarantee
all-or-nothing

The Bank Transfer Problem



Transaction = guarantee BOTH steps succeed, OR NEITHER happens

ACID Properties

A

Atomicity
All or nothing

Transfer: both debit
AND credit, or neither

C

Consistency
Valid state always

Balance never
goes negative

I

Isolation
No peeking mid-txn

No dirty reads from
concurrent users

D

Durability
Survives crashes

COMMIT = permanent
even after power cut

BEGIN TRAN / COMMIT / ROLLBACK

Successful Transfer (COMMIT)

```
BEGIN TRANSACTION;  
  
UPDATE Accounts  
SET Balance = Balance - 1000  
WHERE AccID = 'A';  
  
UPDATE Accounts  
SET Balance = Balance + 1000  
WHERE AccID = 'B';  
  
COMMIT TRANSACTION;  
  
-- Both updates permanent!
```

Failed Transfer (ROLLBACK)

```
BEGIN TRANSACTION;  
  
UPDATE Accounts  
SET Balance = Balance - 1000  
WHERE AccID = 'A';  
  
-- Error! B doesn't exist  
IF @@ERROR <> 0  
    ROLLBACK TRANSACTION;  
  
-- A's debit is UNDONE!  
  
COMMIT TRANSACTION;
```

BEGIN

Updates...

COMMIT

ROLLBACK

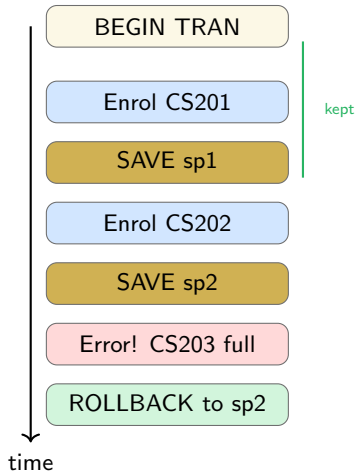
SAVEPOINT — Mid-Transaction Checkpoint

```
BEGIN TRANSACTION;

-- Enrol in Course 1
INSERT INTO Enrolment
VALUES ('S001', 'CS201', GETDATE()
);
SAVE TRANSACTION sp1;  --
    checkpoint!

-- Enrol in Course 2
INSERT INTO Enrolment
VALUES ('S001', 'CS202', GETDATE()
);
SAVE TRANSACTION sp2;

-- Course 3 is full    partial
    rollback
```



Isolation Levels

Level	Description	Dirty Read	Phantom Read
READ UNCOMMITTED	See others' un-committed data	YES	YES
READ COMMITTED	Only see committed data	No	YES
REPEATABLE READ	Rows stable; phantoms possible	No	YES
SERIALIZABLE	Full isolation; strictest	No	No

Dirty Read

T2 reads T1's uncommitted change.

Phantom Read

T2 re-reads a range and finds **new rows**

Real Scenario — Enrol in 3 Courses Atomically

```
BEGIN TRANSACTION;
BEGIN TRY
    -- Enrol in 3 courses      all or nothing
    EXEC EnrolStudent 'S001', 'CS201';
    EXEC EnrolStudent 'S001', 'CS202';
    EXEC EnrolStudent 'S001', 'CS203';

    COMMIT TRANSACTION;  -- all 3 succeeded!
    PRINT 'All 3 enrolments committed.';

END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION;  -- undo everything!
    PRINT 'Error: ' + ERROR_MESSAGE();
    PRINT 'All enrolments rolled back.';
END CATCH;
```


Autocommit Mode

Default: Autocommit ON

```
-- Each statement = own
  transaction
UPDATE Courses
SET Seats = Seats - 1
WHERE CourseID = 'CS201';
-- Auto-committed immediately!
```

- ▶ Simple & safe for single statements
- ▶ Cannot undo after it runs!

When to Use Explicit Transactions

- ▶ Multiple related statements
- ▶ Need all-or-nothing guarantee
- ▶ Financial / critical operations
- ▶ Batch inserts needing consistency

```
-- Turn off autocommit (PostgreSQL)
BEGIN; -- or START TRANSACTION;
...
COMMIT;
```

Common Transaction Pitfalls

Long Transactions — Lock tables too long; blocks other users

Forgetting COMMIT — Changes sit in limbo; locks held

Unchecked ROLLBACK — Silently discards valid work

Nested Transactions — SQL Server: @@TRANCOUNT must reach 0

Best Practices

- ▶ Keep transactions **short**
- ▶ Always handle errors with **TRY...CATCH**

Watch Out

- ▶ Never open transactions in a loop
- ▶ Check @@TRANCOUNT before commit
- ▶ Avoid DDL inside transactions when

Summary

Key Takeaways

- ▶ Transactions = **ACID** guarantee
- ▶ **BEGIN / COMMIT / ROLLBACK**
- ▶ **SAVEPOINT** for partial rollback
- ▶ Isolation levels control **concurrency**
- ▶ Use **TRY...CATCH** for safety



Foundation of reliable databases

Next: L29 — SQL Security (GRANT, REVOKE, SQL Injection)